



南開大學
Nankai University

计算机学院

并行程序设计实验报告

ANN GPU 编程实验

姓名：刘宇泽

学号：2411334

专业：计算机科学与技术

2026 年 6 月

目录

1 实验任务简述	1
2 数据集	1
3 本地实验环境配置	1
4 算法设计	2
4.1 GPU Flat Batch Matrix Baseline	2
4.1.1 原理	2
4.1.2 流程图	2
4.1.3 伪代码	2
4.1.4 复杂度	3
4.1.5 算法分析	3
4.2 GPU IVF Cluster-Union Matrix Baseline	3
4.2.1 原理	3
4.2.2 流程图	4
4.2.3 伪代码	4
4.2.4 复杂度	5
4.2.5 算法分析	5
4.3 GPU IVF Compact Pair Optimization	5
4.3.1 原理	5
4.3.2 流程图	6
4.3.3 伪代码	6
4.3.4 复杂度	7
4.3.5 算法分析	7
5 实验结果对比与分析	7
5.1 GPU Flat Batch 实验结果	7
5.2 GPU IVF Cluster-Union 实验结果	8
5.3 GPU IVF Compact 实验结果	8
5.4 综合对比分析	9
6 Profiling 性能分析	10
6.1 Profiling 方法与数据组织	10
6.2 GPU Flat Batch Profiling 分析	10
6.3 GPU IVF Cluster-Union Profiling 分析	11
6.4 GPU IVF Compact Profiling 分析	11
6.5 Profiling 综合对比	11
7 结论	12
代码仓库	12

1 实验任务简述

单条 query 在 GPU 上难以充分利用硬件并行度，通常需要将多个 query 组成 batch 共同处理。有两个主要方向：第一，将内积距离计算转换为矩阵乘法形式；第二，在 IVF 上进行查询间并行，并分析 batch 内不同 query 的候选簇不一致带来的浪费。

本实验围绕这两个要求实现并测试三类 GPU 算法：

1. **GPU Flat Batch Matrix Baseline:** 将 base 向量矩阵与 query batch 相乘，生成完整距离矩阵，再选取每列 Top- k 。
2. **GPU IVF Cluster-Union Matrix Baseline:** 对 batch 内 query 选择的 IVF 簇取并集，以簇为单位做“簇矩阵 $\times$ query batch”的距离计算，作为 IVF 方向的矩阵 baseline。
3. **GPU IVF Compact Pair Optimization:** 针对 IVF batch 中簇选择不同导致的无效计算，将有效的 ($query, candidate$) 对压平成一维任务，只计算真正需要的候选距离。

2 数据集

实验使用 DEEP100K 数据集，距离定义为内积距离：

$$\delta(x, q) = 1 - \langle x, q \rangle = 1 - \sum_{j=1}^d x_j q_j.$$

检索质量使用 Recall@ k ：

$$\text{Recall@}k = \frac{|R_k(q) \cap G_k(q)|}{k}.$$

表 1: 实验数据集

文件	内容	规模	维度
DEEP100K.base.100k.fbin	base 向量	100000	96
DEEP100K.query.fbin	query 向量	10000	96
DEEP100K.gt.query.100k.top100.bin	ground truth	10000	100

3 本地实验环境配置

表 2: 本地硬件与系统环境

项目	配置	说明
操作系统	Microsoft Windows 11 Pro for Workstations	版本 10.0.26200, 64-bit
CPU	Intel(R) Core(TM) i7-14650HX	16 核 24 线程, WMI 报告主频 2200 MHz
内存	15.71 GB	系统可见物理内存
GPU	NVIDIA GeForce RTX 4060 Laptop GPU	显存 8188 MiB, Compute Capability 8.9
GPU 驱动	NVIDIA Driver 610.47	WDDM 模式, CUDA UMD Version 13.3

表 3: 本地编译、运行与 profiling 工具环境

项目	实际配置	说明
CUDA Toolkit	13.3	安装器实际写入 NVIDIA 默认 Toolkit 目录, 包含 nvcc 13.3
Nsight Systems	2026.1.3	使用 <code>nsys profile</code> 采集 CUDA API、kernel 和 H2D/D2H 拷贝统计
Nsight Compute	2026.2.0	使用 <code>ncu --set basic</code> 对三个 kernel 进行单次 launch 采样
C++ 编译与运行	C++17 / Driver API	实验主程序通过 NVIDIA Driver API 加载 PTX kernel, 算法代码与实验统计均在 C++/CUDA 风格框架中完成

4 算法设计

4.1 GPU Flat Batch Matrix Baseline

4.1.1 原理

设 base data 为矩阵 $B \in \mathbb{R}^{N \times d}$, query batch 为 $Q \in \mathbb{R}^{m \times d}$ 。内积相似度矩阵可写为:

$$S = BQ^T, \quad S_{i,t} = \langle x_i, q_t \rangle.$$

距离矩阵为 $D_{i,t} = 1 - S_{i,t}$ 。在实现中, 为了避免在没有 cuBLAS 的环境中引入额外依赖, baseline 使用手写 kernel: 每个 CUDA thread 计算一个 $(query, base)$ 对应的内积。该方法保留了矩阵乘法 baseline 的核心任务划分, 即将多个 query 合并成 batch, 让 GPU 同时处理 mN 个距离项。

4.1.2 流程图

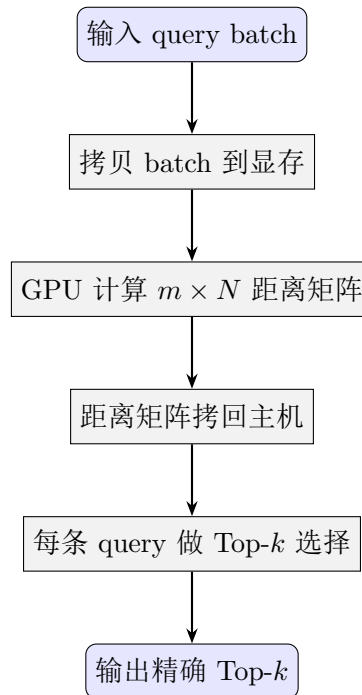


图 1: GPU Flat Batch 检索流程

4.1.3 伪代码

Algorithm 1 GPU Flat Batch Matrix Baseline

Input: base 矩阵 B , query 集合 Q , batch 大小 m , 近邻数 k

Output: 每条 query 的 Top- k 结果

```

1: 将  $B$  上传到 GPU
2: for 每个 query batch  $Q_b$  do
3:   将  $Q_b$  上传到 GPU
4:   启动  $|Q_b| \times N$  个 GPU 线程
5:   for all 线程  $(t, i)$  in parallel do
6:      $D[t, i] \leftarrow 1 - \sum_{j=1}^d B[i, j] Q_b[t, j]$ 
7:   end for
8:   将  $D$  拷回主机
9:   for batch 内每条 query  $t$  do
10:    对  $D[t, :]$  执行 Top- $k$  选择
11:   end for
12: end for

```

4.1.4 复杂度

对于一个 batch, GPU 距离打分需要计算 mN 个内积, 每个内积长度为 d , 因此算术复杂度为 $O(mNd)$ 。距离矩阵需要保存 mN 个浮点数, 显存输出空间与设备到主机传输量均为 $O(mN)$ 。主机侧使用大小为 k 的最大堆维护每条 query 的 Top- k , 选择复杂度为 $O(mN \log k)$, 额外空间为 $O(mk)$ 。该算法不包含索引构建阶段。

4.1.5 算法分析

Flat Batch 是 GPU ANN 实验中的精确基线: 它不牺牲召回率, 能够直接衡量 GPU 对批量内积距离计算的加速能力。算法的并行粒度是单个 $(query, base)$ 距离项, 任务划分规则、负载均衡稳定, 适合验证 batch size 对 kernel launch 开销、显存带宽和 PCIe 传输的影响。其主要局限在于必须生成并回传完整 $m \times N$ 距离矩阵, 后续 Top- k 仍在 CPU 侧完成; 因此当 N 和 m 较大时, 端到端性能会受到结果写回与主机选择阶段约束。

4.2 GPU IVF Cluster-Union Matrix Baseline

4.2.1 原理

IVF (Inverted File) 先通过聚类把 base 向量划分到 $nlist$ 个倒排簇。查询时, 每条 query 只访问与其最接近的 $nprobe$ 个簇。在 GPU batch 场景中, batch 内不同 query 的 $nprobe$ 个簇通常不完全相同。指导书指出, 这会导致矩阵乘法中出现浪费。

Cluster-Union baseline 按照指导书中的 IVF 矩阵思路实现: 对当前 batch 内所有 query 的候选簇取并集 $\mathcal{U}$, 然后对每个簇 $c \in \mathcal{U}$, 将簇内向量写成矩阵 L_c , 计算 $L_c Q_b^\top$ 。如果某条 query 并没有选择簇 c , 该 query 在此簇上的得分会被丢弃, 这部分就是无效计算。实验用

$$\rho = \frac{\text{computed pairs}}{\text{useful pairs}}$$

衡量浪费比。

4.2.2 流程图

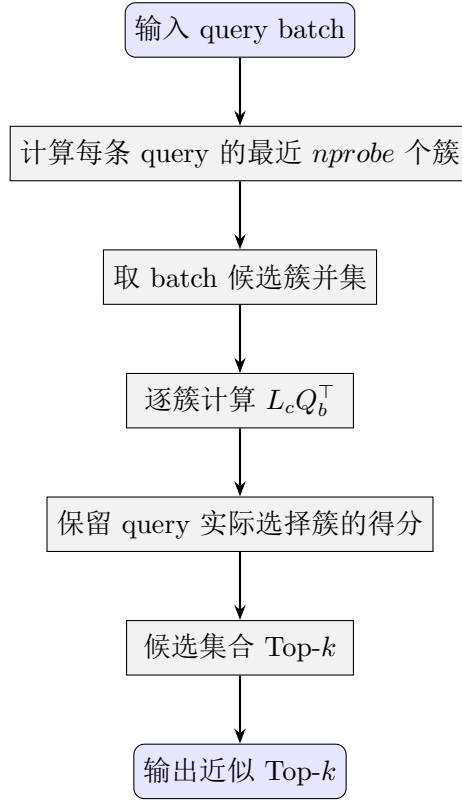


图 2: GPU IVF Cluster-Union baseline 流程

4.2.3 伪代码

Algorithm 2 GPU IVF Cluster-Union Matrix Baseline**Input:** IVF 倒排簇 $\{L_c\}$, centroid 集合 C , query batch Q_b , $nprobe$, k **Output:** 每条 query 的近似 Top- k

```

1: for 每条 query  $q_t \in Q_b$  do
2:    $P_t \leftarrow$  与  $q_t$  最近的  $nprobe$  个 centroid
3: end for
4:  $\mathcal{U} \leftarrow \bigcup_t P_t$ 
5: for 每个簇  $c \in \mathcal{U}$  do
6:   在 GPU 上计算簇矩阵  $L_c$  与整个  $Q_b$  的距离矩阵
7:   for 每条 query  $q_t$  do
8:     if  $c \in P_t$  then
9:       将簇  $c$  中候选及距离加入  $q_t$  的候选集合
10:    end if
11:  end for
12: end for
13: 对每条 query 的候选集合执行 Top- $k$ 
  
```

4.2.4 复杂度

IVF 构建阶段使用 I 轮 k-means，近似复杂度为 $O(I \cdot N \cdot nlist \cdot d)$ ，倒排表空间为 $O(N)$ 。查询 batch 阶段，若并集簇为 $\mathcal{U}$ ，则 GPU 实际计算复杂度为：

$$O\left(\left(m \sum_{c \in \mathcal{U}} |L_c|\right) d\right).$$

真正有用的计算量为：

$$O\left(\left(\sum_{t=1}^m \sum_{c \in P_t} |L_c|\right) d\right).$$

两者的比值即浪费比 ρ 。当 batch 内 query 的候选簇重合度较低时， ρ 会显著大于 1，导致矩阵化计算产生大量无效距离项。

4.2.5 算法分析

Cluster-Union 保留了规则矩阵乘法形态，簇内向量连续访问，kernel 的内存访问模式较为规整，理论上适合进一步替换为 cuBLAS GEMM 或 shared-memory tiling。该方法也直接对应指导书中 IVF-GPU baseline 的思想，便于观察 query 间并行带来的候选簇不一致问题。其核心缺陷是计算集合由 batch 的簇并集决定，而有效集合由每条 query 的 $nprobe$ 个簇决定；当 batch 较大、 $nprobe$ 较大或数据分布导致 query 簇重合度较低时，实际计算量会显著超过必要计算量，端到端性能不一定优于精确 Flat baseline。

4.3 GPU IVF Compact Pair Optimization

4.3.1 原理

IVF Compact 与 Cluster-Union 使用同样的 IVF 索引和同样的 $nlist/nprobe$ 候选策略，但改变 GPU 任务组织方式。主机端先为 batch 内每条 query 收集其实际要访问的候选向量，然后将所有有效的 $(query, candidate)$ 对压平成一维数组：

$$\{(t, i) \mid i \in \bigcup_{c \in P_t} L_c\}.$$

GPU kernel 每个线程只处理一个有效 pair，通过 `candidate_ids` 和 `query_ids` 两个数组定位 base 向量与 query 向量。这样避免了对未被某条 query 选择的簇做无效打分。

4.3.2 流程图

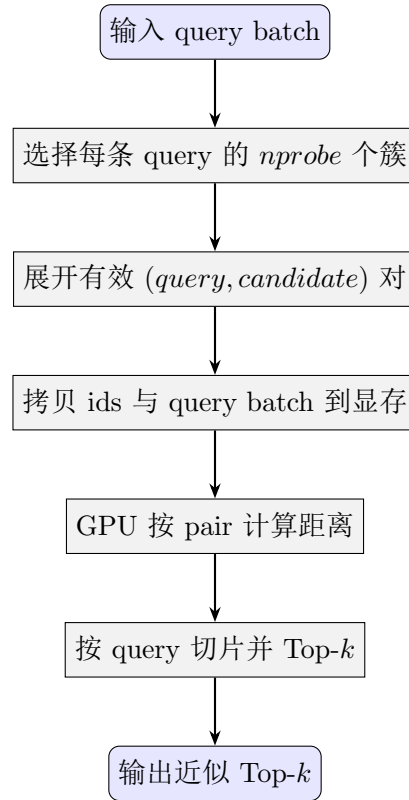


图 3: GPU IVF Compact 流程

4.3.3 伪代码

Algorithm 3 GPU IVF Compact Pair Optimization**Input:** IVF 倒排簇 $\{L_c\}$, centroid 集合 C , query batch Q_b , $nprobe$, k **Output:** 每条 query 的近似 Top- k

```

1: 初始化数组 candidate_ids、query_ids、offsets
2: for 每条 query  $q_t \in Q_b$  do
3:    $P_t \leftarrow$  与  $q_t$  最近的  $nprobe$  个 centroid
4:   for 每个簇  $c \in P_t$  do
5:     for 每个候选  $i \in L_c$  do
6:       追加  $i$  到 candidate_ids, 追加  $t$  到 query_ids
7:     end for
8:   end for
9:   记录  $q_t$  的候选区间到 offsets
10: end for
11: 在 GPU 上每个线程计算一个有效 pair 的距离
12: for 每条 query  $q_t$  do
13:   使用 offsets 切出其候选距离并执行 Top- $k$ 
14: end for

```


4.3.4 复杂度

构建复杂度与 Cluster-Union 相同。查询 batch 的 GPU 打分复杂度为：

$$O\left(\left(\sum_{t=1}^m \sum_{c \in P_t} |L_c|\right) d\right).$$

空间复杂度为有效 pair 数对应的 `candidate_ids`、`query_ids` 和 `distance` 数组。相比 Cluster-Union, Compact 的理论节省比例接近其浪费比 ρ , 但需要额外维护 pair 索引数组, 并引入对 base 向量与 query 向量的间接寻址。

4.3.5 算法分析

Compact 的核心思想是把 IVF 查询中的“按簇矩阵化”改为“按有效 pair 稀疏化”。它牺牲了部分矩阵乘法的规整性, 换取对无效距离项的彻底剔除, 更适合 batch 内 query 候选簇差异较大的场景。该算法的访存连续性弱于 dense GEMM: base 向量通过候选 id 间接读取, query 通过 query id 间接读取, 因此单个距离项的访存效率并非最优; 但在本实验规模下, 减少无效计算和结果传输带来的收益明显大于间接访问开销, 是延迟与召回折中最优的 GPU 方案。

5 实验结果对比与分析

5.1 GPU Flat Batch 实验结果

表 4: GPU Flat Batch 的 batch size 与数据规模实验 ($k = 10$)

query 数	batch	Recall@10	平均延迟/us	kernel 合计/ms	D2H 合计/ms
2000	32	0.999950	389.493	405.709	122.553
2000	64	0.999950	379.778	394.729	119.594
2000	128	0.999950	375.686	397.341	115.142
4000	64	0.999975	377.838	789.747	231.807

在 2000 条 query 上, batch size 从 32 增加到 128 后, 平均延迟从 389.493 us/query 降到 375.686 us/query, 改善幅度有限但方向稳定。原因是该实现中每个距离项独立计算, 计算部分已经足够并行, 后续完整距离矩阵拷回与主机堆选择占据了较大比例。query 数从 2000 增加到 4000 时, batch=64 的平均延迟从 379.778 us/query 变为 377.838 us/query, 说明 Flat Batch 的吞吐在该规模区间比较稳定。

表 5: GPU Flat Batch 的 k 参数实验 (query=2000, batch=64)

k	Recall@ k	平均延迟/us	P95/us	候选数
1	1.000000	384.086	410.703	100000
10	0.999950	379.778	399.814	100000
20	0.999975	379.909	406.466	100000
50	1.000000	395.212	414.355	100000

改变 k 对 GPU kernel 时间几乎没有影响，因为距离计算仍然覆盖全库。端到端时间差异主要来自主机侧堆选择和运行波动；当 $k = 50$ 时，堆维护成本略有上升，平均延迟增加到 395.212 us/query。该结果说明 Flat Batch 的主要瓶颈不是 k ，而是完整距离矩阵的写回和主机侧 Top- k 。

5.2 GPU IVF Cluster-Union 实验结果

表 6: GPU IVF Cluster-Union 参数实验 (query=2000, batch=16, $k = 10$)

$nlist$	$nprobe$	Recall@10	平均延迟/us	候选数	浪费比 ρ	kernel/ms
64	4	0.9091	534.651	8783.9	8.088811	513.348
64	8	0.9680	377.067	16218.9	5.456155	362.906
128	8	0.9445	397.801	9101.7	7.819867	350.127

当 $nprobe$ 从 4 增大到 8 时，Recall@10 从 0.9091 提高到 0.9680，候选数从 8783.9 增加到 16218.9。该 baseline 的浪费比仍然明显，三组参数均在 5.45 以上，其中 $nlist = 64, nprobe = 4$ 达到 8.09。平均延迟不只由候选数决定，还受簇并集规模、单簇大小和 kernel launch 次数影响；但浪费比清楚说明 batch 内候选簇不一致会产生大量最终不被使用的距离计算，这正是指导书中强调的 IVF-GPU 问题。

5.3 GPU IVF Compact 实验结果

表 7: GPU IVF Compact 参数与数据规模实验 ($k = 10$, batch=32)

query 数	$nlist$	$nprobe$	Recall@10	平均延迟/us	候选数	kernel/ms
2000	32	4	0.934350	88.520	14895.3	52.314
2000	64	8	0.968000	96.130	16218.9	56.942
2000	64	16	0.991550	180.781	29141.9	133.244
2000	128	8	0.944450	96.915	9101.7	81.058
4000	64	8	0.967350	172.367	16109.1	326.093
4000	128	8	0.945925	223.801	9050.6	304.325

$nprobe$ 从 8 增加到 16 时，Recall@10 从 0.968000 提高到 0.991550，同时平均延迟从 96.130 us/query 增加到 180.781 us/query，说明候选数是 IVF 查询时间的主导因素之一。 $nlist = 128, nprobe = 8$ 的候选数下降到 9101.7，召回也下降到 0.944450； $nlist = 64, nprobe = 8$ 在召回和延迟之间更均衡。

数据规模从 2000 条 query 增加到 4000 条 query 后， $nlist = 64, nprobe = 8$ 的 Recall@10 基本稳定，但平均延迟从 96.130 us/query 上升到 172.367 us/query；这主要来自 4000 条实验中候选 id、距离结果和主机堆选择的累计开销增加。与 Cluster-Union 在 $nlist = 64, nprobe = 8$ 下的结果相比，Compact 在 2000 条 query 上保持相同 Recall@10=0.968000，但平均延迟从 377.067 us/query 降到 96.130 us/query，约快 3.92 倍。这说明对 IVF batch 进行有效 pair 压缩可以直接解决簇并集 baseline 的无效计算问题。

5.4 综合对比分析

表 8: 代表性 GPU 实验结果（正式实验均从 2000 条 query 起）

算法	query	k	Recall	平均延迟/us	候选数	说明
GPU Flat Batch	2000	10	0.999950	379.778	100000	精确矩阵 baseline
GPU Flat Batch	4000	10	0.999975	377.838	100000	数据规模扩展
GPU IVF Compact	2000	10	0.968000	96.130	16218.9	有效 pair 优化
GPU IVF Compact	4000	10	0.967350	172.367	16109.1	数据规模扩展
GPU IVF Union	2000	10	0.968000	377.067	16218.9	簇并集 baseline

从召回率、平均延迟、候选规模和浪费比四个维度看，三类算法体现出不同的性能边界。GPU Flat Batch 是精确检索，Recall@10 接近 1，且在 2000 到 4000 条 query 的规模变化中平均延迟基本稳定，说明 batch 化确实能够充分摊薄 kernel launch 开销并维持较高 GPU 吞吐。然而该算法对每条 query 均访问全部 100000 个 base 向量，候选规模和距离矩阵传输量固定为全库规模，因此它更适合作为精确性能基线，而不是低延迟 ANN 方案。

IVF Cluster-Union 保持了接近矩阵乘法的规则计算结构，但实验结果表明，规则性并不必然带来端到端最优性能。以 $nlist = 64, nprobe = 8$ 为例，其 Recall@10 与 Compact 方法同为 0.968000，但平均延迟为 377.067 us/query，接近 Flat Batch 的 379.778 us/query。原因在于 batch 内 query 的候选簇并不一致，簇并集计算引入了浪费比 $\rho = 5.456155$ 的额外距离项。这说明在 IVF-GPU 中，候选簇一致性是决定矩阵化 baseline 是否有效的关键条件。

IVF Compact 的实验结果构成了本实验中更优的召回-延迟折中。仍以 $nlist = 64, nprobe = 8$ 为例，它在 2000 条 query 上保持 Recall@10=0.968000，同时平均延迟降至 96.130 us/query，相比 Cluster-Union 加速约 $377.067/96.130 \approx 3.92$ 倍，相比 Flat Batch 加速约 $379.778/96.130 \approx 3.95$ 倍。该结果说明，当 IVF 的候选集合天然稀疏且 query 间簇重合度不足时，将任务从“簇矩阵 $\times$ query batch”改写为有效 pair 计算，可以显著降低无效算术、设备到主机传输和主机侧选择压力。

参数层面， $nprobe$ 是最直接的精度-效率调节量。 $nlist = 64$ 时， $nprobe$ 从 8 增加到 16，Compact 的 Recall@10 从 0.968000 提高到 0.991550，但候选数从 16218.9 增至 29141.9，平均延迟也从 96.130 us/query 增至 180.781 us/query。 $nlist$ 的作用更复杂：增大 $nlist$ 会缩小单簇规模并减少候选数，但如果 $nprobe$ 不同步增大，候选覆盖范围下降，召回率可能降低。因此 IVF 参数调优不能只追求更小候选数，而应结合目标 Recall、候选簇覆盖率和 GPU 批处理开销共同确定。

从瓶颈来源看，当前实现仍将 Top- k 选择放在主机侧，GPU 主要承担距离打分。这使得 Flat Batch 受到完整距离矩阵回传限制，Compact 受到候选 id 构造、候选距离回传和主机堆选择限制。若继续优化，可将 per-query Top- k 移至 GPU 端，减少 D2H 数据量；对 Flat Batch 可进一步使用 cuBLAS GEMM 或 shared-memory tiling；对 IVF Compact 可研究候选 pair 的按 query 分组压缩、GPU 端分段 Top- k 和 pinned memory/stream 重叠传输，以进一步降低端到端延迟。

6 Profiling 性能分析

6.1 Profiling 方法与数据组织

采用 NVIDIA 推荐的新工具链完成 profiling: `nsys` 用于全程序 CUDA API、kernel 和 Host-Device 拷贝统计, `ncu` 用于单个 kernel 的吞吐率、occupancy 和 launch 配置分析。

```

1 nsys profile --trace=cuda,nvtx --stats=true --force-overwrite=true \
2   -o data/gpu_experiments/profiling/nsys_ann_quick \
3   ./gpu/gpu_ann_main.exe --out data/gpu_experiments/profiling/quick_run --quick
4
5 ncu --set basic --target-processes all --kernel-name flat_score_kernel \
6   --launch-count 1 -o data/gpu_experiments/profiling/ncu_flat_score_kernel \
7   ./gpu/gpu_ann_main.exe --out data/gpu_experiments/profiling/ncu_run --quick

```

需要说明的是, `ncu` 会通过 replay 多次执行被测 kernel, 因此单 kernel duration 更适合分析微架构状态; 端到端时间判断仍以普通运行 CSV 和 `nsys` 汇总为主。

表 9: Nsight Systems 的 kernel 运行时间统计

kernel	总时间/ms	占比/%	launch 次数	平均/us
flat_score_kernel	411.290	51.7	32	12852.812
cluster_score_kernel	329.498	41.4	6364	51.775
indexed_score_kernel	54.446	6.8	63	864.226

表 10: Nsight Compute 的单 kernel 采样指标

kernel	Mem/%	DRAM/%	SM/%	Occ./%	Reg	Block	Grid	Threads
flat	97.64	71.67	12.03	83.40	24	256	25000	6400000
cluster	92.49	4.64	11.47	76.86	27	256	235	60160
indexed	85.67	22.89	10.67	93.98	27	256	1805	462080

6.2 GPU Flat Batch Profiling 分析

Flat Batch 对应的核心 kernel 为 `flat_score_kernel`。在 quick profiling 中, 该 kernel 共启动 32 次, Nsight Systems 统计的 GPU kernel 总时间为 411.290 ms, 平均每次 launch 约 12.853 ms。该 kernel 的单次任务规模较大: 以 batch size 64 为例, 一个 launch 需要计算 64×100000 个 query-base 距离项, 表 10 中对应的 grid size 为 25000、线程数为 6400000。

Nsight Compute 显示 `flat_score_kernel` 的内存吞吐率达到 97.64%, Achieved Occupancy 为 83.40%, 而 SM 计算吞吐率约为 12.03%。这说明 Flat Batch 的瓶颈并不主要来自浮点乘加能力不足, 而是来自全库向量读取、距离矩阵写回以及后续 D2H 传输。结合算法结构看, 每条 query 都必须访问全部 100000 个 base 向量, 并生成完整距离矩阵, 因此该方法虽然规则、负载均衡且召回率最高, 但其端到端延迟天然受内存带宽和结果回传约束。

6.3 GPU IVF Cluster-Union Profiling 分析

Cluster-Union 对应的核心 kernel 为 `cluster_score_kernel`。该 kernel 在 quick profiling 中启动 6364 次，单次平均只有 51.775 us，但累计 kernel 时间达到 329.498 ms。与 Flat Batch 相比，Cluster-Union 的单个 kernel 更小，但 launch 次数远高于 Flat Batch；这说明其主要性能问题不是单次距离计算过慢，而是 batch 内簇并集拆分后产生了大量细粒度 kernel 和同步事件。

从 Nsight Compute 指标看，`cluster_score_kernel` 的内存吞吐率为 92.49%，Achieved Occupancy 为 76.86%。虽然簇内 base 向量访问相对规整，但由于每个 batch 内 query 选择的 IVF 簇并不完全一致，Cluster-Union 需要对簇并集内的向量与整个 query batch 做矩阵化计算，最终产生浪费比 $\rho = 5.456155$ 的无效距离项。profiling 结果与实验结果相互印证：该方法保留了矩阵计算的规整形式，却被簇并集导致的无效计算、频繁 launch 和 D2H 回传抵消了收益。

6.4 GPU IVF Compact Profiling 分析

Compact 对应的核心 kernel 为 `indexed_score_kernel`。该 kernel 在 quick profiling 中启动 63 次，Nsight Systems 统计的总 kernel 时间为 54.446 ms，平均每次 launch 约 864.226 us，明显低于 Flat Batch 和 Cluster-Union 的总 kernel 时间。该结果说明，把 IVF 查询改写为有效 (*query, candidate*) pair 后，GPU 实际执行的距离项规模被显著压缩。

Nsight Compute 显示 `indexed_score_kernel` 的 Achieved Occupancy 达到 93.98%，高于 Flat Batch 和 Cluster-Union；但其内存吞吐率为 85.67%，低于 Flat Batch。这一现象符合 Compact 的算法特征：candidate id 和 query id 的间接寻址降低了访存规整性，但它剔除了无效候选簇计算，因此总体 kernel 时间更低。换言之，Compact 不是通过单个距离项更快取胜，而是通过减少实际距离项数量和减少 D2H 数据规模获得端到端优势。

6.5 Profiling 综合对比

表 11: Nsight Systems 的 Host-Device 拷贝统计

方向	总时间/ms	次数	平均/us	总量/MB	平均/MB	最大/MB
Device-to-Host	191.015	6459	29.573	1637.693	0.254	25.600
Host-to-Device	41.032	6711	6.114	344.452	0.051	38.400

从三个算法的 profiling 结果看，Flat Batch 是“少量大 kernel”的模式，单次 kernel 运行时间长，内存吞吐率高，适合作为精确矩阵 baseline；Cluster-Union 是“大量小 kernel”的模式，单次 kernel 短但 launch 次数过多，并且存在明显簇并集浪费；Compact 是“中等数量有效 pair kernel”的模式，通过压缩实际计算集合，使总 kernel 时间最低。

从通信角度看，D2H 总时间 191.015 ms、总数据量 1637.693 MB，明显高于 H2D 的 41.032 ms 和 344.452 MB。这说明当前实现的共同瓶颈在于距离结果需要回传到 CPU 侧完成 Top-*k* 选择。Flat Batch 回传完整距离矩阵，Cluster-Union 回传包含浪费项的簇并集距离结果，Compact 回传有效 pair 距离结果，因此 Compact 在数据回传维度也更占优。后续若继续优化，应优先将 per-query Top-*k* 或分段 Top-*k* 下沉到 GPU 端，并结合 pinned memory 与 stream 重叠传输降低 D2H 开销。

7 结论

本实验按照 GPU 编程指导书中 ANN 选题要求，完成了矩阵乘法方向的 batch baseline、IVF 方向的簇矩阵 baseline，以及针对 IVF batch 浪费问题的 compact pair 优化。所有主函数、算法函数、索引构建、参数扫描和 CSV 写出均为 C++ 实现，正式实验均从 2000 条 query 起。实验表明，GPU 对 ANN 距离计算的加速必须依赖 query batch；单纯保持矩阵形式并不一定最优，尤其在 IVF 中，batch 内候选簇差异会带来显著无效计算。通过压缩有效 (*query, candidate*) 对，GPU IVF Compact 在保持较高召回的同时显著降低端到端延迟，是当前实验中最有效的 GPU 优化方案。

代码仓库

本实验相关代码已整理至 GitHub 仓库：[NKU-cpu-parallel-lab/lab5_GPUprogramming](https://github.com/NKU-cpu-parallel-lab/lab5_GPUprogramming)